

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО» (Университет ИТМО)

**Исследовательский центр «Сильный искусственный интеллект в
промышленности»**

Институт искусственного интеллекта

ОТЧЁТ

о прохождении стажировки

Тема: Разработка многомодульной RAG-системы с иерархическим
выбором инструментов для управления городской инфраструктурой
на основе LLM-агентов

Выполнил: Федорин К.В.

Научный руководитель: Никитин Н. О.

СОДЕРЖАНИЕ

Введение	3
1 Обзор деятельности Исследовательского центра	5
1.1 Исследовательский центр «Сильный искусственный интеллект в промышленности»	5
2 Разработка многомодульной RAG-системы с иерархическим выбором инструментов	6
2.1 Архитектура иерархического выбора инструментов	6
2.1.1 Уровень 1: Агент-кондуктор и выбор пайплайна	6
2.1.2 Уровень 2: Специализированные агенты	6
2.2 Механизм валидации	8
2.3 Реализация	8
3 Разработка промптов для LLM-агентов	10
3.1 Методология	10
3.2 Промпты для выбора инструментов	10
3.3 Промпты для валидации	11
3.4 Промпты для генерации ответа	11
4 Обеспечение качества (CI/CD)	12
4.1 Непрерывная интеграция	12
4.2 Автоматизация тестирования	12
4.3 Поддержание качества кода	12
4.4 Методология	12
4.5 Результаты	13
4.6 Анализ	13
Заключение	14
Список использованных источников	15

ВВЕДЕНИЕ

Отчёт этот составлен по итогам стажировки в Исследовательском центре «Сильный искусственный интеллект в промышленности» (сокращённо — СИИП). Центр работает на базе Университета ИТМО. Его организовали в 2021 г. в рамках федерального проекта «Искусственный интеллект»; в 2025 г. он снова выиграл грантовый конкурс — что, в общем, говорит об уровне исследований и разработок, которые здесь ведутся.

Стажировка проходила в лаборатории, где занимаются сильным ИИ для промышленных задач. Одно из ключевых направлений — построение систем поддержки принятия решений на больших языковых моделях (Large Language Models, LLM) и мультиагентных архитектурах. Такие системы находят применение в управлении городским хозяйством, промышленной автоматизации, стратегическом планировании.

Цель стажировки формулировалась так: получить практические навыки по всему циклу создания мультиагентных LLM-систем. Сюда входило и проектирование иерархического механизма выбора инструментов, и разработка промптов, и организация CI/CD, и экспериментальная проверка готового решения.

Работа велась в рамках исследовательского проекта. Его результатом стала публикация «LLM Agents for Smart City Management: Enhancing Decision Support Through Multi-Agent AI Systems» в журнале Smart Cities (MDPI, ISSN 2624-6511, том 8, выпуск 1, статья 19, 2025 г.). Журнал индексируется в Scopus и Web of Science, квартиль Q1 [1]. В статье описывалась архитектура многомодульной системы, объединяющей RAG с иерархическим порядком выбора инструментов; эта архитектура прошла экспериментальную проверку.

За время стажировки пришлось решать несколько задач:

- спроектировать и запрограммировать механизм вложенного выбора инструментов для LLM-агентов — по сути, иерархический RAG с каскадным перебором вызовов;

- написать системные и пользовательские промпты для агентов каждого уровня;
- настроить CI/CD для автоматизации тестирования и контроля качества;
- провести бенчмаркинг решения по ряду метрик.

1 Обзор деятельности Исследовательского центра

1.1 Исследовательский центр «Сильный искусственный интеллект в промышленности»

Центр основали на базе Университета ИТМО в 2021 г. в рамках федерального проекта «Искусственный интеллект» (под эгидой Минэкономразвития). Дважды, в 2021 и 2025 гг., Центр выигрывал грантовый конкурс — это позволило ему войти в число семи ведущих исследовательских центров страны по ИИ.

Миссия, как её формулируют в Центре, — сформировать научно обоснованные каноны сильного ИИ применительно к промышленности и создать методическую базу для прорывных цифровых решений, востребованных и в России, и за рубежом. Спектр технологий, которые здесь разрабатывают, довольно широк: от фреймворков автоматического машинного обучения (FEDOT.Industrial, iOpt) и библиотек анализа поведения (OCEAN AI) до средств тематического моделирования (AutoTM) и интеллектуальных систем поддержки принятия решений на LLM (ProtoLLM). В проекте выдвигались и проверялись три гипотезы. Согласно первой из них, LLM-агенты должны быть способны маршрутизировать и обрабатывать разнородные запросы, касающиеся городской среды и социнфраструктуры. Вторая заключалась в том, что применение RAG совместно с LLM повышает качество ответов — особенно при работе с локальными нормативами и специфическими городскими данными. Третья предполагала, что интеграция LLM с действующими городскими сервисами (доступность социальных благ, транспортная доступность и т.п.) даёт более точные и контекстуально релевантные ответы. Итогом проекта стала публикация в Smart Cities (Q1) [1], где описана архитектура и результаты экспериментов.

2 Разработка многомодульной RAG-системы с иерархическим выбором инструментов

По мере работы выяснилась знакомая вещь: чем длиннее список доступных инструментов, тем хуже модель справляется с выбором. В рамках стажировки для решения этой проблемы применили иерархический (вложенный) подход. Если коротко — задачу выбора разбили на два уровня. На верхнем агент-кондуктор определял, какой пайплайн обработки запустить, на нижнем — специализированные агенты внутри этого пайплайна подбирали конкретные функции для извлечения контекста.

2.1 Архитектура иерархического выбора инструментов

2.1.1 Уровень 1: Агент-кондуктор и выбор пайплайна

На верхнем уровне помещался агент-кондуктор. Ему передавался полный запрос пользователя; через механизм вызова инструментов (tool calling) агент определял тип запроса и выбирал одну из двух веток:

1) **service_accessibility_pipeline** — для запросов, связанных с доступностью городских сервисов и объектов инфраструктуры (образование, здравоохранение, культура, спорт, ЖКХ, демография и пр.); 2) **strategy_development_pipeline** — для запросов, ответ на которые следовало искать в документах стратегического развития города (здесь применялся RAG).

Для каждого инструмента готовили описание, куда закладывали не только общее назначение, но и примерные ключевые слова, по которым модель могла бы ориентироваться. Для первого пайплайна это были «доступность», «обеспеченность», «количество», для второго — «планируется», «мероприятия». Если модель не могла уверенно выбрать, по умолчанию срабатывал пайплайн стратегического развития — такое поведение было самым безопасным.

2.1.2 Уровень 2: Специализированные агенты

После выбора пайплайна управление передавалось агентам второго уровня; они уже занимались детальным подбором функций.

2.1.2.1 Пайплайн доступности городских сервисов

Здесь работал специализированный агент вызова инструментов (Tool Calling Agent) с доступом к девяти функциям, возвращающим статистику по разным категориям:

- **get_general_stats_education** — сведения об образовательных учреждениях (детсады, школы, вузы);
- **get_general_stats_healthcare** — данные о здравоохранительных учреждениях (поликлиники, больницы, аптеки);
- **get_general_stats_culture** — культурно-досуговые объекты (библиотеки, музеи, театры);
- **get_general_stats_sports** — спортивные объекты (бассейны, фитнес-центры, стадионы);
- **get_general_stats_services** — объекты сферы услуг (магазины, банки, парикмахерские);
- **get_general_stats_demography** — демографические показатели;
- **get_general_stats_housing_and_communal_services** — показатели ЖКХ;
- **get_general_stats_complaints** — жалобы на городские и жилищные услуги;
- **get_general_stats_provision** — обеспеченность городскими службами (пожарные, полиция, транспорт);
- **get_general_stats_recreation** — рекреационные зоны и объекты отдыха.

У каждой функции было развёрнутое описание — какие данные возвращает, для каких запросов подходит. Агент второго уровня отбирал от одной до трёх наиболее подходящих функций. Если модель не могла определить нужные — срабатывал механизм значений по умолчанию.

Помимо этого, для учёта типа территории (город, муниципалитет, район, квартал) применялся эвристический метод `define_default_functions`, добавлявший базовые функции соответственно типу. То есть, для запроса по

конкретному району сразу подтягивались релевантные ему статистические показатели.

2.1.2.2 Пайплайн стратегического развития

Второй пайплайн реализовывал более классическую RAG-схему. Запрос направлялся в векторную базу ChromaDB, где ищались фрагменты документов стратегического развития Санкт-Петербурга. Для векторизации применялась мультязычная модель Multilingual-E5-large. Из базы извлекались четыре наиболее релевантных отрывка, которые вместе с исходным вопросом передавались LLM-агенту для формирования ответа.

Дополнительный выбор инструментов здесь не требовался — источник данных был фиксированным.

2.2 Механизм валидации

Помимо двухуровневой архитектуры, для повышения точности разрабатывался механизм валидации. После того как LLM принимала решение, специальный агент-валидатор проверял его корректность: получал исходный запрос, описания выбранных функций и ответ LLM, после чего выносил вердикт. При ошибке он возвращал название правильной функции.

В конечной конфигурации этот механизм сделали опциональным — от него отказались ради сокращения времени ответа. Но архитектура валидации была проработана и реализована полностью; в будущем её можно задействовать без существенных доработок.

2.3 Реализация

Программная реализация велась на Python с применением FastAPI для организации микросервисов. В системе можно выделить несколько ключевых компонентов.

Первый — модуль Agent (agents/agent.py), базовый класс с методами для взаимодействия с LLM: `get_relevant_functions` (отправка запроса), `parse_function_names_from_agent_answer` (разбор ответа), `choose_functions` (выбор) и `check_choice_correctness` с `check_functions` (проверка корректности).

Ещё один важный компонент — модули описания инструментов (agents/tools/pipeline_tools.py и agents/tools/accessibility_tools.py). Они

содержали формализованные описания в формате OpenAI function calling: имя функции, её назначение, схема параметров.

Третий — модули самих пайплайнов (`pipelines/master_pipeline.py`, `accessibility_pipeline.py`, `strategy_pipeline.py`), где сосредоточена логика обработки запросов на каждом уровне.

Четвёртый — модуль промптов (`agents/prompts.py`), хранящий системные и пользовательские промпты для всех агентов.

Система спроектирована масштабируемой: добавление нового источника данных сводится к созданию инструмента с описанием и реализации соответствующего пайплайна.

3 Разработка промптов для LLM-агентов

3.1 Методология

Работа над промптами велась с учётом нескольких принципов. Во-первых, однозначность: промпт должен явно задавать формат ответа и границы дозволенного. Во-вторых, ролевая настройка: для каждого агента определялась его функция (кто-то выбирал инструменты, кто-то валидировал, кто-то генерировал ответ). В-третьих, компактность: избыточная информация способна запутать модель. В-четвёртых, контекстуальная изоляция: агенты разных уровней получали только те описания инструментов, которые относились к их уровню — не больше.

3.2 Промпты для выбора инструментов

Для агента, выбирающего функции, был составлен системный промпт `fc_sys_prompt`, задающий ролевую модель и перечисляющий доступные функции:

```
fc_sys_prompt = r"""You are a helpful assistant with access to
the following
functions. Use them if required - ${tools}."""
```

Пользовательский промпт `fc_user_prompt` предписывал модели возвращать исключительно имена функций, разделённые пробелами — без пояснений:

```
fc_user_prompt = r"""Extract all relevant data for answering
this
question: ${question}
You MUST return ONLY the function names separated by spaces.
Do NOT return any other additional text."""
```

Для агента-кондуктора, которому нужно было выбрать один из двух пайплайнов, написали `binary_fc_user_prompt` — модель должна вернуть ровно одно имя:

```
binary_fc_user_prompt = r"""Extract all relevant data for
answering this
question: ${question}. You MUST return ONLY the function name.
Do NOT return any other additional text."""
```

3.3 Промпты для валидации

Для валидатора разрабатывались два промпта: для проверки выбора внутри пайплайна доступности (ac_cor_user_prompt) и для проверки выбора самого пайплайна (pip_cor_user_prompt). Оба содержали инструкцию: сравнить выбранную функцию с исходным запросом и списком доступных, при ошибке — вернуть правильное наименование. Формат ответа строгий: «[Correct answer]: правильная функция».

```
ac_cor_user_prompt = r"""[Instruction]: You are given a
question, functions
descriptions and an answer. Your task is to compare the chosen
function with
the question and the descriptions and determine if the function
was selected
correctly. If the chosen function is correct, return the
function name.
If the function is selected incorrectly, return the name of the
correct
function. [Question]: ${question}. [Answer]: ${answer}.
[Function Descriptions]: ${tools}.
You MUST return ONLY the correct functions in this format:
[Correct answer]: correct function. Do NOT return Context."""
```

3.4 Промпты для генерации ответа

Агент, формирующий финальный ответ на основе извлечённого контекста, получал отдельные промпты для каждого из двух источников. Промпты содержали правила: использовать только предоставленный контекст, не добавлять сведений из собственных знаний, ссылаться на источники. Полные тексты — в приложении к статье [1].

4 Обеспечение качества (CI/CD)

4.1 Непрерывная интеграция

Параллельно с разработкой функциональности разворачивалась система непрерывной интеграции и доставки на GitHub Actions. Её задача — поддерживать стабильность кодовой базы, автоматизировать тестирование и снижать вероятность регрессий.

Был подготовлен workflow «Test build and launch app». Он запускается при создании или обновлении pull request в ветки develop и main, плюс ежедневно по расписанию (03:25 UTC). Процесс включал извлечение кода из репозитория, настройку переменных окружения для подключения к API городских сервисов, сборку Docker-образа и запуск тестового скрипта.

Параметры, которые нельзя было светить (токены, URL эндпоинтов), держали в GitHub Secrets и подставляли в окружение непосредственно на раннере — так они не просачивались в репозиторий.

4.2 Автоматизация тестирования

Тестовый сценарий был написан на bash: собирал Docker-образ, запускал контейнер и проверял доступность API-эндпоинтов. Такой подход давал интеграционное тестирование — проверялась не только сборка, но и связность всех компонентов: LLM-сервиса, векторной базы, API городских сервисов.

4.3 Поддержание качества кода

Помимо автоматизированных проверок, в течение стажировки велась работа по контролю качества кода в целом. Проводилось рецензирование pull request коллег, поддерживался единый стиль, отслеживалась актуальность документации. Эти меры помогали удерживать базу в работоспособном состоянии на всём протяжении разработки.

4.4 Методология

Для оценки предложенного решения организовали бенчмаркинг. Измеряли точность выбора пайплайна (доля запросов, где агент-кондуктор верно определил источник данных), точность выбора функций (доля запросов с корректно подобранными функциями), качество ответа по

метрике G-Eval [2], а также сравнивали результаты с ответами модели, работающей без RAG и инструментов.

Эксперименты ставились на наборе из 150 пар «вопрос-ответ», покрывающих разные аспекты городского управления. Для маршрутизации и генерации применялись GPT-4o, Llama 3.1 (70B), Mixtral 8x22B.

4.5 Результаты

Мультиагентная система показала точность выбора пайплайна 94–99% для разных моделей. Подключение RAG повысило точность ответов на 17% по запросам стратегического развития и на 55% по вопросам доступности услуг. Комбинированный режим (документная база + API сервисов) дал наивысшие показатели G-Eval: 0,68–0,74 против 0,30–0,38 у standalone LLM.

Выяснилось также, что для хороших результатов не обязательно использовать самые крупные модели: Llama 3.1 70B показывала сопоставимые с GPT-4o результаты и по точности маршрутизации, и по качеству ответов.

4.6 Анализ

Анализ подтвердил все три гипотезы. Иерархический подход к выбору инструментов показал себя работоспособным: разбиение на два уровня позволило агенту-кондуктору оперировать всего двумя альтернативами вместо десятка, что заметно снизило нагрузку на модель. Специализированные агенты второго уровня имели дело с ограниченным набором функций.

Было отмечено, что при росте числа инструментов точность выбора LLM падает — это лишний раз подтверждает целесообразность иерархической организации. В статье это зафиксировано как одно из направлений будущих исследований.

ЗАКЛЮЧЕНИЕ

За время стажировки в Центре «Сильный искусственный интеллект в промышленности» Университета ИТМО удалось получить практический опыт полного цикла разработки мультиагентной LLM-системы поддержки принятия решений для городского управления.

Конкретные результаты таковы:

- спроектирована и реализована архитектура иерархического выбора инструментов: на верхнем уровне агент-кондуктор выбирает пайплайн, на нижнем — специализированные агенты определяют конкретные функции;
- разработаны системные и пользовательские промпты для всех уровней — для выбора, валидации и генерации ответов;
- развёрнут CI/CD на GitHub Actions, автоматизирующий сборку, тестирование и проверку работоспособности;
- проведён бенчмаркинг, подтвердивший точность выбора пайплайна 94–99% и существенное повышение качества ответов при использовании RAG и иерархического подбора инструментов.

Результаты легли в основу статьи «LLM Agents for Smart City Management: Enhancing Decision Support Through Multi-Agent AI Systems» (Smart Cities, MDPI, квартиль Q1).

Накопленный опыт и разработанные решения могут найти применение в дальнейших исследованиях мультиагентных LLM-систем — для промышленной автоматизации, стратегического планирования и других областей, где требуется интеллектуальная поддержка принятия решений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Kalyuzhnaya A. и др. LLM Agents for Smart City Management: Enhancing Decision Support Through Multi-Agent AI Systems // Smart Cities. 2025. Т. 8, № 1. С. 19.
2. Liu Y. и др. G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment [Электронный ресурс]. 2023. URL: <https://arxiv.org/abs/2303.16634> (дата обращения: 29.06.2026).